< input is in file
> fout - overwrite file
>> fout - append to existing file
cmd1|cmd2 → pipe

[a-zA-Z][0-9]
^ - line beginning, $ - end of line
\< - start of word, \> end
* - 0 or more apparitions
+ - 1 or more apparitions
? - 0 or 1 apparitions
\s - blank character
(abc) - group, | - or

grep -q (quiet)
-c: count nr de lines
-v: only those not matching
-n: display matching lines + the line no.
-r: recursively
-i: case insensitive
-x: match exactly the entire line
-o: only the part that matches

sort -r: reverse
-u: uniq
-M: compares lines

sed -f: from a file
p: print | d - delete
i: insert (before)
a: append (after)
s/regex/string/flags(opt)
\n

awk - if, for, while = C
· BEGIN{ }/END{ }
· NF - no. of fields $1, $2...
· NR -
· $0 - entire line
· FS - field separator
· length(string)
· split (string, array, separ.)
· index(s1, s2) ⇒ strstr

wc -l - lines
-w - words; -c - characters

shell | !/bin/bash
- chmod +x file
if ... [redacted] then ...; fi
$ var - access the value
for var in ... do ... done
" " - interpreted (spec. chars)
' ' - luate ca atare
$"var1" = $"var2"
test -lt, -le, -gt, -eq, -ge, -ne
test -z (string), -n (string)

shell | $((expr)) integer
`command` - executes command and replaces it with the result of the execution
shift - shift arguments to the left
$# - argc, $@ - argv

UNIX - interactive, time-sharing, multi-user, multi-tasking
KERNEL - nucleum (cod unic) loaded at the PC startup
System CALL: kernel mode to user mode for access to disk, network, video mem.
KERNEL MODE = level 0 protection
USER MODE = level 3 protection
levels 1,2 = services

FILE TYPES: regular, directory, hard/soft links, fifo, socket
SYMBOLIC LINKS: in-is target name
- behave as the target file/directory
- can reference on another disk
- delete file - points to a file that doesn't exist
HARD LINKS: In target file
- new i-node for the same f.
- have to be
- file is deleted only when there are no more links to it
ROOT (superuser): BOSS

METODE DE ALOCARE A MEM
① SINGLE TASKING:
- one single program, to open another one, we need to close this one
② MULTITASKING:
Ⓐ Real:
a) absolute: i) fixed partitions
⊕ more procs
⊖ the max dim of proc
⊖ program compiled on a specific partition
ii) partiții relocabile: the addresses are stored as OFFSETS inside a partition
b) dynamic partitions:
fragmented memory ⇒ memory needs to be compressed (compactată?) (close processes): all of it, or as much as needed for one process
↳ SOLUTION: paged memory

Ⓑ VIRTUAL: a) paged
- virtual addresses are generated in the form of (page, offset), each [redacted] proc. has a table of pages
address: offset + pf * dim - page
b) segmented: code + data segments grouped in
· the access to the seg. is controlled by the OS
the seg. can be reentrant (entered from another program)
⊖ fragmenting
c) paged + segmented ♥
· segments are divided into more virtual pg.
the proc. has a table of segments, each of ↓ has a table of pages

LOADING POLICIES
① All pages in the beginning
⊕ fast access at runtime
⊖ slow startup
⊖ loading pages that are not necessarily needed
② each page when needed
⊕ fast startup
⊕ loading only what is needed
⊖ slow access at runtime

LOCATION PRINCIPLE | the pages neighbouring the page that has just been loaded are more likely to be needed in the future, so we load them preemptively

REPLACING POLICIES
· how to choose the victim pg
FIFO: each pg. has an age, the oldest ones go first
NRU: each pg. has two bits (r, w) that are set to 1 at r/w and the OS is periodically setting them to 0 ⇒ classes (0,0), (0,1), (1,0), (1,1)
↓ first ... second ... → last recently used
LRU - matrix of N×N, N = no. of pg.; when accessing pg. i, line i is populated with 1, column ... 0; pages with the lesser 1s are first to go

PIPE | pipe(int p[2])
0 - read, 1 - write

FIFO | mkfifo(char* name, int mode)
empty - read blocks until no more data
full - write block until all data is read → space
O_NDELAY → ret 0 if fifo empty / full
No process to read/write:
- O_NDELAY ⇒ error
- normal ⇒ wait
- procs need not be related

STATES OF A PROCESS
Hold →1 Ready ⇄2/3 Run →8 Fin; Swap 4/7; Run →5 wait; wait →6 Ready
1 - create proc, 2 - get CPU, 3 - release CPU, 4 - move on disk, release CPU, 5 - i/o or wait for resources, 6 - op. from 5 done, free disk space (dealocate memory), 8 - release CPU, free mem, destroy process
** - alloc. mem
* - free mem.

PLACEMENT POLICIES
· how to handle malloc?
- FIRST FIT:
⊕ fast ⊖ no attempt to control fragmentation
- BEST FIT
⊕ economic in terms of large memory areas/chunks
⊖ slower ⊖ creates small areas of free memory ⇒ fragmentation
- WORST FIT
⊕ leaves behind larger chunks
⊖ slower
- BUDDY → always alloc. a chunk of the smallest power of two greater than the required size; keep lists of free chunks by powers of 2; alloc the largest available chunk and distribute the remaining memory in pow of 2 chunks
⊕ speed

WAIT - suspends the exec until a child finishes (any, or a specific one)
EXEC - replace the whole image with the given command
EXIT - exits the current proc and goes back to the parent

DEADLOCK
- proc/thread enters an ∞ waiting state because it needs a resource that is held by another
CAUSES OF DL:
1. Mutual exclusion when accessing resources
2. Hold and wait: block a res. while waiting for another
3. Non-preemption: resources cannot be "stolen"
4. Circular wait
SOLUTION TO DL
- reboot → pick a victim proc → create a backup state
DETECTION OF DL
graf: □ proc, o res.
→ is held by, --→ is needed
cycle ⇒ deadlock
PREVENTION OF DL
→ pick an order for res. and always ask in that order

ZOMBIE → process that has finished execution but still has an entry in the proc table; use wait to avoid or before creating: signal(SIGCHLD, SIG_IGN)
ORPHAN = still executes, parent is dead

SCHEDULING ALGO
① FIRST COME, FIRST SERVED
② SHORTEST FIRST:
⊖ we need to know durations
⊖ not always right
③ Priority Based
⊖ starvation
④ Deadline Scheduling: Scheduling based on the finish time (needs to finish f...)
⑤ Round Robin: time sharing, ... procs are working simultaneously, ... READY, one → R...

SEMAPHORE | (value) with two operations
P(S) - wait for pr(A)
v(S) --
if(v(S) <0)
state(A) = wait
q(S) ← A /A w...
pass ctrl to sys
else pass ctrl to A

SEMAPHORE (value, queue)

V(S) - SIGNAL for A:

```
V(S)++
if (V(S) <= 0)
    q(S) → B
    // B comes through
    state(B) = Ready
    pass ctrl to sys. scheduler
else
    pass ctrl to A
```

Process vs. Thread

P contains one or more T

⊕ quicker to create T > P

⊕ quicker to switch between T

⊕ T share data easily

⊖ no security between T

⊖ one T blocks ⇒ all T in that P block

UNIX [illegible] commands [illegible]

UNIX filesystem structure

Block	Content
0	Boot block
1	Super block
2 … m	Inode blocks
m+1 …	Data blocks

⊕ a kind of map for ...

FOR I-NODES

- A0: for the first 10 blocks of 512 bytes of the file
- A1: indirectare simplă (the next 128 blocks of 512 bytes)
- A2: indirectare dublă ($128^2 * 512$)
- A3: indirectare triplă ($128^3 * 512$) $(B/A)^m$

CACHE ① Direct Cache: store the page in location page_addr % cache_size

⊕ fast ⊕ simple

⊖ cache trashing (collisions)

② Associative Cache

place each page on a free location found through iterating the cache ⊖ slow

CACHE ③ Set Associative Cache

- hash table; caching in sets of pages
- the set of a page is calc. as page_addr % cache_set, then a free location is found iterating through the set

⊕ flexibility ⊕ avoid col.

⊖ doesn't use all available cache lines

SIGNAL: sets a function to be called for a certain signal (RUTINĂ DE TRATARE)

• does not signal itself.

MOUNT: associates a storage device to a particular location in the directory tree

int dup(int desc_vechi)

int dup2(int d_vechi, int d_nou)

Read la FIFO: waits until there is no proc open for writing or until there is data, returns the read data or 0 if ∄ a proc to write

int open(char* name, int flag, [int rights])

O_RDONLY, O_WRONLY, O_RDWR, O_APPEND, O_CREAT, O_TRUNC, O_NDELAY (FIFO)

read/write (handle, buff, nr_bytes)

WAIT → RUN?

↳ program loaded into memory waiting for CPU (OS scheduler decides)

RUN → WAIT

↳ additional cond.

How many threads to process 1M files? A: the number of available cores (cores of the processor)

Why do I/O ops cause a proc to move from state RUN to Wait?

A: The proc has to pause itself while it gets/sends the data through the I/O and it frees the proc in the meantime for other procs.

How is the address calculation done in the absolut fixed part?

A: multiple fixed part. and the RAM locations are est. first prog > first portion, first add → beginning of part

Line containing [illegible] and not 6

grep "^[illegible]" [illegible] | grep -v 6

fifos

A -x→ B -y→ C -z→ A

```
if (a==0){
    open("x", O_RDONLY);
    open("y", O_WRONLY);
}
if (b==0) ... x, y
if (c==0) ... z, y
```

2 sets of 4 vs 4 sets of 2

at max 3 vowels

grep -E ^(.*[aeiou]){3}.*$

4 rules that specify the no of reps

+ → 1 ≤

* → 0 ≤

? → 0/1

{n} → n times

FIFO for reading, proc? → wait, while [illegible] opens the [illegible]

[illegible] pipe → DON'T forget to close the proc

Hard-link only in the same partition → when creating a hard link, the OS creates a new directory entry that points to the existing inode of the file; it works seamlessly in the same partition bc. the inode is unique in that part, the OS cannot assign that inode no. to it in a diff. part, because part have unique inodes within them.

at least x vowels/ numbers

grep -E "^[^]*[aeiou][^]*){n,}"

having a group

grep -E "^([^:]*:){4}[^:]*\ion\>"

all odd no of chars

grep -E "^(..)*.$" a.txt

swap neighbouring digits

sed -E 's/([0-9])([0-9])/\2\1/g'

d1 d2 swap global

multiple of binary 4

grep -E '^([01]*0{2}){2}.*' | grep -E '^[01]{5,}$'

5 digits